Continuous Assessment Portfolio: Submission Document

Student: Praise Samuel okoro
Email / Student ID: p.okoro4197@miva.edu.ng
Institution: Miva Open University, School of Post Graduate Studies
Programme: Master of Information Technology (MIT)
Session: 2026/2027 Academic Session, First Semester Continuous Assessment
Submission channels: Miva LMS (this document) and a private GitHub repository
GitHub repository: https://github.com/psamuel32/advanced-database-portfolio

1. The Organisation I Selected as My Case Study

I built the whole portfolio around one case study that I chose myself: ApexPay Global, a multi-currency core digital banking and instant settlement platform. I picked a payments platform because it gave me enough variety to practise everything the course asked for, from strict relational design through to a distributed, always-on deployment. ApexPay serves retail customers and commercial merchants, holds multi-currency deposit accounts in NGN, USD, EUR and GBP, moves money between accounts as atomic peer-to-peer transfers over a double-entry ledger, authorises card payments, and screens transactions for financial crime. I sized it at roughly 4.5 million ledger entries a day across about 2.45 million customers and 3.1 million accounts, with around 120 GB of new data each month and a read-to-write mix of about 70 to 30.

When I wrote down who would actually use the database, I ended up with five groups: retail and SME customers who check balances, pull statements and start payments; commercial merchants who accept payments and receive daily settlement; AML and compliance analysts who investigate suspicious activity and circular money movement; finance and audit staff who reconcile the double-entry ledger and reverse bad transfers; and database reliability engineers who run replication, sharding, failover and backups.

The main data I modelled was customer identity and KYC records, multi-currency deposit accounts, the immutable double-entry ledger, tokenised card credentials and fraud surveillance telemetry. The operations I wanted the database to support were atomic multi-account transfers, idempotent payment handling, continuous ledger reconciliation, real-time velocity checks, automatic reversal when a gateway times out, graph-based detection of mule rings, and zero-RPO multi-region replication.

2. Portfolio 1: My Database Design and Modelling

I designed seven relations in PostgreSQL 16: customers, accounts, transactions, ledger_entries, merchants, cards and fraud_alerts. I used UUIDv4 surrogate primary keys so account and customer identifiers cannot be enumerated, put unique constraints on customer email and phone, and wired the foreign keys with ON DELETE RESTRICT so a customer or account with balance history cannot be deleted and orphan the ledger. I pushed the two rules that matter most down to the database kernel itself with CHECK (balance >= 0) and CHECK (amount > 0), plus ISO currency domain checks, so an invalid row cannot be committed even if the application layer is bypassed. I documented the entity relationship model with a full attribute dictionary and Crow's foot cardinality.

I normalised the schema step by step, starting from a deliberately messy reporting export and working up to Boyce-Codd Normal Form. The first normal form removed the repeating customer and multi-account groups; the second removed the partial dependencies on the composite ledger key; the third removed the transitive dependency account_id to customer_email to the customer attributes by splitting customers away from accounts; and I proved BCNF by showing that every non-trivial determinant is a superkey.

I kept the design and the proof in 01_schema_ddl.sql, 02_erd_specification.md and 03_normalisation_proof.md, and captured the live table and constraint output in evidence/p1_tables.txt and evidence/p1_accounts_definition.txt.

3. Portfolio 2: My Query Processing and Optimisation Work

I wrote three production queries for the case study and benchmarked them against 28.4 million ledger entries and 14.2 million transactions: a running balance reconciliation using a window function over the ledger, a 30-minute sliding-window fraud velocity check using DENSE_RANK with a HAVING filter, and an end-of-day merchant settlement aggregation. I captured every plan with EXPLAIN (ANALYZE, BUFFERS, VERBOSE, FORMAT JSON), once before I added any index and once after.

Query	Before indexing	After indexing	Change
Q1 Running balance	Sequential scan on 28.4M ledger rows, 22,766.5 ms, 347,552 disk blocks	Index scan on (account_id, posted_at, entry_id), 4.19 ms, 12 disk blocks	5,436x faster
Q2 Fraud velocity	Sequential scan on 14.2M transaction rows, 3,591.9 ms, 233,493 disk blocks	Bitmap index scan on my partial index, 233.1 ms, 79 disk blocks	15x faster
Q3 Merchant settlement	Hash join over sequential scans, 4,445.1 ms, 240,507 disk blocks	Merge join with index access, 1,647.4 ms, 30,385 disk blocks	2.7x faster

What I took from these results is that the first two indexes worked because they matched the query closely. The composite index matched the window function's partition and ordering exactly, and the partial index matched the fraud query's predicate exactly, so in both cases the planner stopped reading the whole table. The settlement query taught me the opposite lesson: it only improved 2.7x, because the query still has to look at a full day of transactions and the planner kept a merge join, so my covering index cut heap reads without removing the scan. I also measured the cost of indexing, and the extra indexes on transactions reduced write throughput by roughly 22 percent.

I kept my queries, plans and analysis in 01_advanced_queries.sql, 02_execution_plans.json and 03_indexing_benchmarks.md, with the raw before-and-after plans in evidence/p2_plans_baseline.txt and evidence/p2_plans_indexed.txt.

4. Portfolio 3: My Transactions and Concurrency Work

For the commit path I modelled a transfer of 500.00 with a 2.50 fee. I lock the rows with SELECT ... FOR UPDATE, debit the sender by 502.50, credit the recipient by 500.00, credit the platform fee account by 2.50, post three balanced ledger entries, and commit synchronously.

For the rollback path I deliberately let a sender with too little money attempt a transfer. The database raises ERROR 23514, a check_violation on accounts_balance_check, and the transaction rolls back with the balances untouched. I liked this test because it proves the balance rule is enforced by the kernel and not only by my application code.

For concurrency I worked through dirty reads, non-repeatable reads and lost updates against the ANSI SQL-92 isolation levels, and checked what I actually saw against PostgreSQL's MVCC tuple visibility, namely xmin and xmax. My early transfers running in both directions at once deadlocked occasionally, and I fixed that by always locking accounts in ascending account_id order, which makes a circular wait impossible.

I kept the scripts and write-up in 01_acid_transfer_commit.sql, 02_rollback_fault_injection.sql and 03_isolation_concurrency_study.md, with the captured sessions in evidence/p3_commit_output.txt and evidence/p3_rollback_output.txt.

5. Portfolio 4: My NoSQL and Advanced Data Models

Here I set out to show why a single relational store is not the right answer for every part of this system, and I built three complementary stores on the same case study. I used MongoDB, the document store, for the card-network webhooks and KYC documents, because their shape changes from one network and issuer to the next and a document model absorbs that far better than a wide, sparse relational table. I used Redis, the key-value store, for session revocation and API rate limiting, because those counters are short-lived and only need to be fast and atomic, which I implemented as a Lua script. I used Neo4j, the graph database, for detecting circular mule rings, because representing accounts as nodes and transfers as relationships lets a three-hop cycle (account to account to account and back) be found by pointer traversal, where the same question in SQL needs recursive self joins that get slower with every hop.

The reason I chose each store came down to fit: documents for variable payloads, key-value for ephemeral counters that need speed more than durability, and a graph for relationship questions that are expensive to express in SQL. I recorded those choices and the CAP trade-offs in my trade-off matrix.

I kept the models and analysis in 01_mongodb_webhook_events.json, 02_redis_rate_limiter.lua, 03_neo4j_aml_detection.cypher and 04_polyglot_tradeoff_matrix.md, with the captured output in evidence/p4_mongodb_load.txt, evidence/p4_mongodb_aggregation.txt, evidence/p4_redis_output.txt and evidence/p4_neo4j_output.txt.

6. Portfolio 5: My Distributed and Cloud Database Work

I designed the distributed layer to reach 99.999 percent availability with no committed data loss. I put the read and write primary in europe-west3-a with a synchronous in-region standby in europe-west3-b, which gives an RPO of 0 seconds, and added asynchronous read replicas in us-east1 and asia-northeast1 to serve balance queries close to the user. I hash sharded transactions and ledger entries on account_id and colocated the ledger with its account so most joins run locally without distributed two-phase commit, and I used monthly range partitions on transactions for pruning and data tiering. For high availability I configured Patroni to run leader election over an etcd quorum with fencing against

split brain, targeting an RTO under 15 seconds. When I reasoned through a partition I decided the ledger should favour consistency over availability and refuse writes without a quorum, and in normal operation writes pay a synchronous round trip while reads are served locally.

The evidence I captured shows the sharding working: I registered all seven relations, distributed accounts and transactions on account_id with ledger entries colocated, and confirmed the coordinator's shard layout. I want to be honest that the multi-region topology is a designed and documented artefact, in the topology and Patroni configuration files, rather than a deployment I could run on a single local machine.

I kept the design in 01_replication_sharding_topology.md, 02_patroni_cluster_config.yaml, 03_citus_sharding_setup.sql and 04_cap_pacelc_resilience_study.md, with the captured output in evidence/p5_citus_sharding.txt, evidence/p5_pg_dist_shard.txt and evidence/p5_citus_nodes.txt.

7. How I Organised My Repository and Evidence

I followed the structure the brief asked for:

- portfolio-1-database-design/
- portfolio-2-query-optimisation/
- portfolio-3-transactions-concurrency/
- portfolio-4-nosql/
- portfolio-5-distributed-cloud/
- evidence/
- README.md

I wrote a README in each portfolio folder so that anyone opening it can see what the scripts do, how to run them and what output to expect. I put the raw captured output in the evidence folder, including the DDL verification, the before-and-after execution plans, the transaction and rollback sessions, the NoSQL output, and the Citus cluster and shard layout, so the claims I make are backed by output rather than assertion. I built the repository up with commits spread across the term instead of one upload at the end.

8. My Individual Technical Reflection

Looking back, the decision that shaped everything else was choosing synchronous replication for the in-zone standby. It adds a network round trip of about 4 milliseconds to every commit, and I accepted that cost because it is the only thing I looked at that guarantees zero committed data loss if a datacentre fails. The hardest problem I ran into was locking: my early transfers running in both directions at once deadlocked now and then, and I felt a real sense of relief when sorting the account identifiers before locking removed every deadlock without me weakening the isolation level or adding retry loops.

The lesson that stayed with me most is about indexing. I went in assuming an index is always an improvement, but the settlement query barely moved because the day-range scan was still there, and the extra indexes quietly cost me about 22 percent of write throughput. If I keep building on this, I would replace my fixed-window rate limiter with a measured sliding-window one, benchmark the write path under sustained load, and prove the multi-region failover by injecting a partition instead of only documenting it.

9. References

1. A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York: McGraw-Hill, 2020.

2. PostgreSQL Global Development Group, PostgreSQL 16 Documentation: Concurrency Control and Index Types, 2024. https://www.postgresql.org/docs/16/

3. M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA: O'Reilly Media, 2017.

4. D. J. Abadi, Consistency Tradeoffs in Modern Distributed Database System Design: PACELC, IEEE Computer, vol. 45, no. 2, pp. 37-42, 2012.

5. Neo4j, Inc., The Neo4j Cypher Manual, 2024. https://neo4j.com/docs/cypher-manual/current/

6. MongoDB, Inc., MongoDB Manual: Aggregation Pipeline, 2024. https://www.mongodb.com/docs/manual/

7. Miva Open University, MIT 8103 Advanced Database Systems Course Continuous Assessment Specification, School of Post Graduate Studies, 2026.

10. My AI Use Declaration

I used AI tools as learning aids while preparing this portfolio, and I want to be clear about how.

The tools I used: Google AI Studio (Gemini) to produce a first draft of the artefacts, and an AI coding assistant afterwards for line-by-line review and correction.

What I used them for: generating an initial draft of the schema, queries, transaction scripts, NoSQL models and distributed configuration, and then helping me review and correct those drafts.

How I reviewed, corrected and verified the output: I read and checked every artefact myself. That review caught and fixed, among other things, an invalid UUID literal in my transfer script, a Cypher statement that referenced variables outside their scope, a partial index predicate that did not match the query it was written for, a Citus distribution key that would have been rejected at creation time, and corrupted notation in my normalisation proof. I took the performance figures in this report straight from the captured EXPLAIN (ANALYZE, BUFFERS) output in my evidence folder, and I corrected the benchmark documentation where the numbers there did not match that captured output. I also re-checked the functional dependencies against relational algebra and the constraints against the DDL.

I understand that using these tools does not remove my responsibility for the accuracy, originality and understanding of the work I am submitting, and that responsibility remains mine.